COMP2511

Tutorial 4

Last Week's Tutorial

- Testing, Debugging, Exceptions
- Domain Modelling

This Week's Tutorial

- Design Principles
- Design by Contract
- Design Patterns
- Strategy Pattern

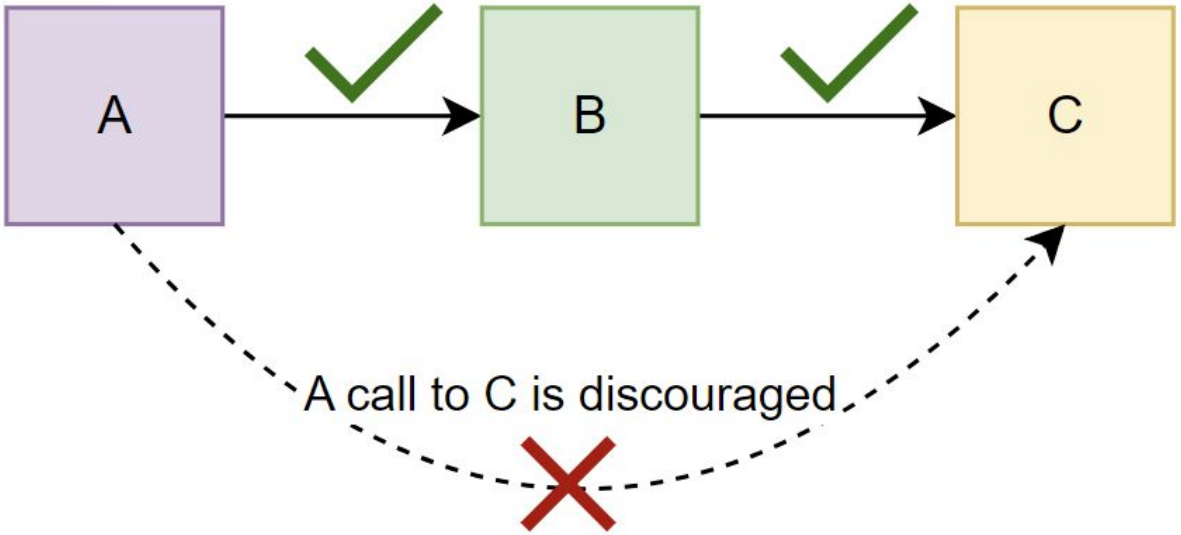
SOLID Principles

- A set of five design principles that are intended to make object-oriented systems more understandable, flexible and maintainable.
  - **S**ingle Responsibility Principle: Each class should have one specific role, and perform only that role.
  - **O**pen-Closed Principle: Entities should be open for extension, but closed for modification.
  - **L**iskov Substitution Principle: Subclasses should be valid instances of their superclass; substituting them for their parent class should not alter the program's correctness.
  - **I**nterface Segregation Principle: An interface should only prescribe methods that are relevant to the classes that will implement it (i.e. make bite-sized interfaces).
  - **D**ependency Inversion Principle: All modules (classes) should depend on abstractions.

The Law of Demeter/Principle of Least Knowledge

- Objects should only interact with their immediate dependencies, and assume as little as possible about anything else in the system.
- This means that classes should only talk to its ‘neighbours’, rather than any neighbours of neighbours.
- In particular, abiding by this principle helps to avoid **tight coupling** by minimising the number of classes affected by changes to another class.
- Violations of this principle *usually* look something like `x.getA().getB()`.
    - This is referred to as **message chaining**. It is typically indicative of some sort of underlying issue with how responsibilities have been delegated across classes (some classes not doing enough or equivalently some classes trying to do too much).

The Law of Demeter/Principle of Least Knowledge



When is it OK for a class to use another class' methods?

- Within the method of a class, it is allowed to use:
  - its own other methods
  - methods of objects that are fields of the class
  - methods of objects that are passed in as parameters
  - methods of objects that are created within the method
- Classes should NOT use the methods of objects returned by another method.
- In most Law of Demeter violations, the underlying issue is that responsibilities haven't been handled in the appropriate classes.
  - Doing `A a = x.getA(); B b = a.getB();` instead of `x.getA().getB()` is NOT a Law of Demeter fix - consider why x needs to get B from A in the first place.

Live Example: Design Principles

- In **src/training**, there is some skeleton code for a training system.
  - Trainers are hosting seminars, with the restriction that each seminar cannot have more than 10 attendees.
  - An attendee can attend a seminar if they are available on the date of the seminar.
- In the TrainingSystem class, there is a method to book a seminar for an employee given the dates on which they are available. This method violates the principle of least knowledge (Law of Demeter).
  - **Refactor the code so that the principle is no longer violated.**

Design by Contract

- **Design by Contract** is an approach where code is written around a set of clear and well-defined specifications, which *when adhered to* guarantees correct and expected behaviour.
  - For example, a specification could say that null will not be given as an input, so you would not have to account for this in code.
  - This can make a program's logic clearer, but potentially vulnerable.
- This contrasts **defensive programming**, which tries to account for unforeseen circumstances and unexpected inputs or user actions.
  - This involves a lot of error-checking which makes programs more secure and robust. However, this could result in a lot of "filler" code that obscures the main logic of the program.

Design by Contract

- **Terminology.** In the context of a method,
  - a **precondition** is a condition on the **input** that the method will perform as expected.
  - a **postcondition** is a guarantee on what the method will **output** or do, *as long as the precondition holds*.
  - an **invariant** is a property of the system that is guaranteed to be maintained once the method has been performed.
    - this property can break *during* a method's execution, but it must be restored once the method is completed.
- If a method's precondition is not satisfied when it is executed, you can no longer make any assumptions about the method's correctness.

The Liskov Substitution Principle

- The **Liskov Substitution Principle** (LSP) states that everywhere an instance of a superclass is used, an instance of its subclass should be able to be used.
- With DbC, contracts are inherited from superclasses to their subclasses.
- This means that:
  - **preconditions** in a subclass must either stay the same as the superclass or be **weakened**.
  - **postconditions** in a subclass must either stay the same as the superclass or be **strengthened**.

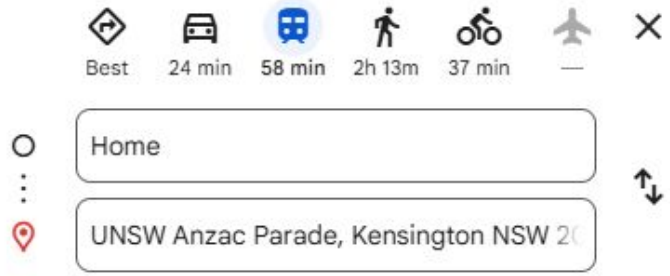
Design by Contract and LSP

- Suppose B is a subclass of A. Is the inheritance in the following examples valid?
  - A has a method with a precondition that it accepts integers between 0-50. B overrides the method to accept any integer between 0-100.
    - This is **valid**. Any input that satisfies the preconditions of A's method also satisfies the preconditions of B's method.
    - The other way around would be invalid!
  - A has a method with a postcondition that it returns a sorted list of integers. B overrides the method to return an unsorted list.
    - This is **invalid**. An unsorted list is not a sorted list, so the postcondition is not satisfied by B's method.
    - The other way around would be valid!

Design Patterns

- Design Patterns are typical repeatable solutions to common problems in software design.
  - They are basically the ‘tried and true’ solutions to specific problems, and are widely recognisable to other programmers.
  - When used properly, they reduce the risk of introducing design flaws and provide flexibility and extensibility to object-oriented systems.
- They are not immediate solutions themselves - they represent **templates** that you need to know how to adapt according to the scenario.
  - Design patterns are useful, but don’t use them for the sake of using them - always consider if the pattern can actually be applied for a good reason.
- **Refactoring Guru**: a great resource that has explanations and examples for each of the design patterns covered in the course.

Hypothetical Scenario...

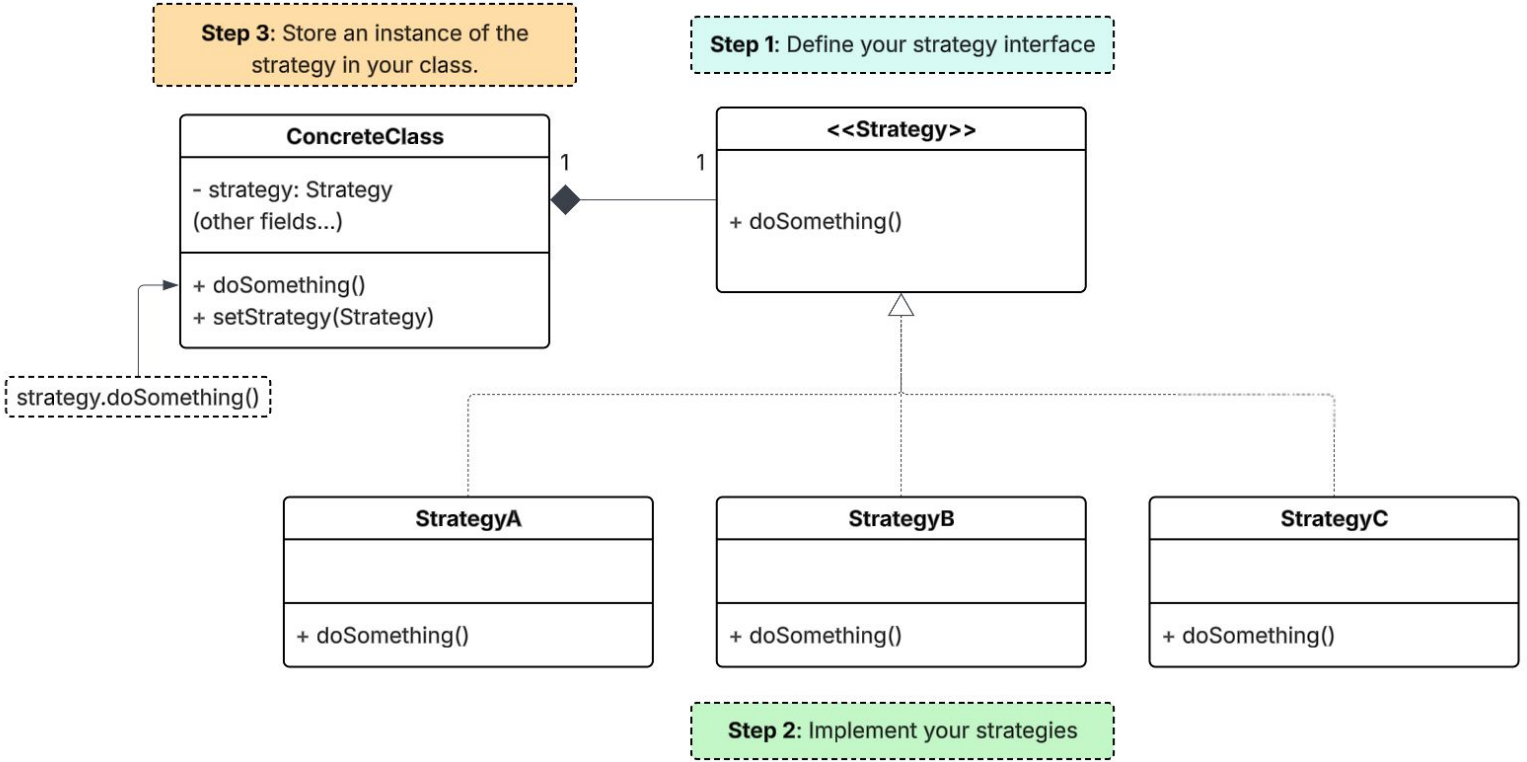


- Consider how you can alternate between different modes of transport while planning a route on Google Maps.
  - The specific details of the route, e.g. the distance you have to cover, the turns you have to take, any costs incurred etc. may be very different across the modes of transport.
  - Regardless, the common aspect of each of these routes is that they all go the same destination.
  - You can think of the transport options as being “different ways to do the same thing”, and you can easily switch between them. This is the main idea behind the **Strategy Pattern**.

Strategy Pattern

- The Strategy Pattern is a design pattern where multiple implementations are defined for a specific action, and a class is able to switch between these implementations at **run-time** (while the programming is running).
- Rather than placing all of the different implementations in a single class, each implementation is separated into different classes called **strategies**.
- The class stores a strategy as a field - due to *polymorphism*, the behaviour of the object will depend on the specific strategy being stored.

Strategy Pattern: Implementation



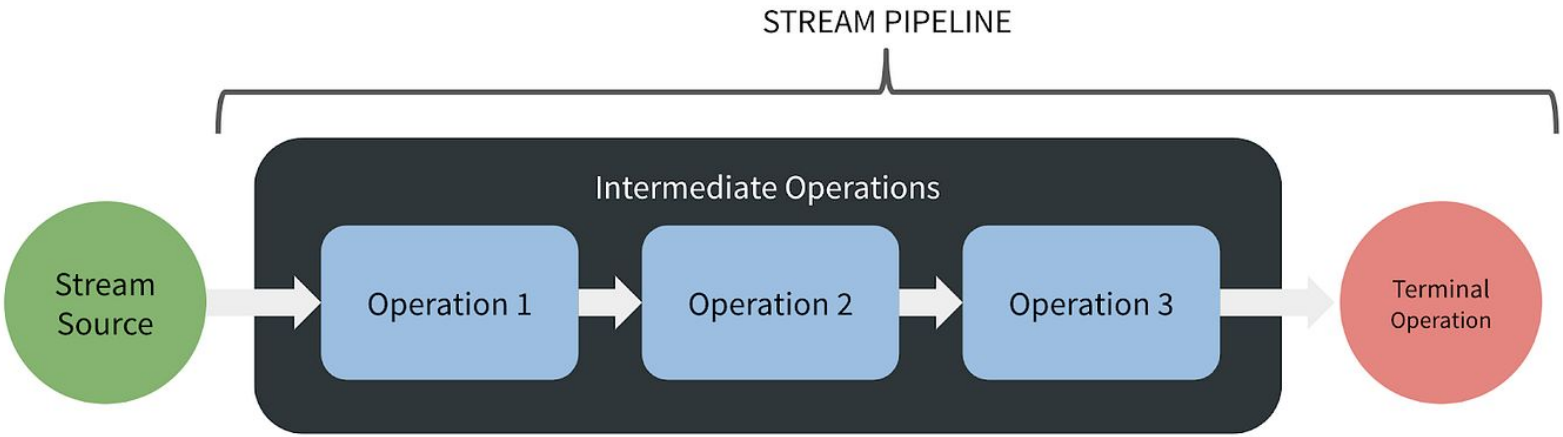
Live Example: Strategy Pattern

- Inside **src/restaurant** is a restaurant payment system with the following requirements:
  - The restaurant has a menu (a list of meals). Each meal has a name and price.
  - Users can enter their order as a series of meals, and the system returns their cost.
- The prices on meals vary in different circumstances. The restaurant has three different price settings (so far):
  - **Standard** - normal rates
  - **Holiday** - 15% surcharge on all items for all customers
  - **Happy Hour** - registered members get a 40% discount, standard customers get 30%
- Refactor the code using the Strategy Pattern to handle the three settings, then implement a new **Prize Draw** Strategy - every 10th customer (since the start of the promotion) gets their meal for free!

Streams

- A stream is an abstracted sequence of elements. Collections in Java (eg. `List`, `Map`, `Set`) can be transformed into streams using `.stream()`.
- Elements in a stream can only be interacted with using particular methods - these work specifically on streams, and provide abstractions.
- Two types of stream operations:
  - **Intermediate** operations: transforms the stream into another stream. These operations can be chained together.
    - `map`, `filter`, `distinct`
  - **Terminal** operations: marks the end of a stream pipeline and produces a final result.
    - `reduce`, `sum`, `toList`

Streams


STREAM PIPELINE
Intermediate Operations
Stream Source
Operation 1
Operation 2
Operation 3
Terminal Operation

Stream Operations

- `filter`: only keeps elements that satisfy a specific condition.
  - `.filter(x -> x < 5)`
- `map`: applies a function to all elements in the stream.
  - `.map(x -> x * 3)`
- `reduce`: combines elements into a single result.
  - `.reduce(1, (a, b) -> a * b)`
- `distinct`: removes duplicate elements.
- `toList`: convert stream into a list (returned list is **immutable**).

```
List<Integer> l = List.of(1, 2, 3, 4, 5, 6);
l = l.stream().filter(x -> x < 5).map(x -> x * 3).toList();
```

```
[3, 6, 9, 12]
```

Live Example: Streams

- Complete the tasks in **src/stream** using streams.